KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY

Course No: CSE 4224

Course Title: Digital System Design Laboratory

Automated Pet Feeding Scheduler

Submitted By

Faiyaz Mahmud **S.M. Shible Sadik**
Roll No: 2007103 Roll No: 2007120

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna–9203, Bangladesh

April, 2026

Contents

1 Introduction **1**
1.1 Background 1
1.2 Motivation 1
2 Objectives **1**
2.1 Primary Objective 1
2.2 Specific Objectives 1
3 System Architecture **2**
3.1 Control Unit 2
3.2 Memory Unit 3
3.3 Arithmetic Logic Unit (ALU) 3
4 Finite State Machine Design **5**
4.1 FSM States 5
4.2 State Transition Logic 6
5 Input and Output Signals **7**
6 Reset and Constraint Mechanisms **7**
7 Hardware Implementation **8**
8 Testing and Verification **8**
9 Limitations and Future Work **9**
10 Conclusion **9**

Introduction

Background

Embedded automation is becoming an integral component of modern smart home systems. Pet care, traditionally dependent on manual intervention, can benefit greatly from scheduled automation. Feeding animals at irregular or delayed intervals can lead to health problems, and existing mechanical feeders lack flexibility or real-time scheduling capability. Field-Programmable Gate Arrays (FPGAs) present an ideal platform for such applications: they offer deterministic, hardware-level timing control, low power consumption, and configurable I/O interfaces, all within a compact programmable device.

This project implements a fully hardware-based Automated Pet Feeding Scheduler on an FPGA using Verilog HDL. Rather than delegating timing logic to a microcontroller executing sequential code, the system employs synthesized digital circuits that guarantee cycle-accurate behavior.

Motivation

Microcontroller-based solutions for pet feeding typically use software delays or interrupt-driven clocks, which can suffer from jitter and software faults. In contrast, an FPGA-based design replaces software timekeeping with a synthesized clock-divider chain derived directly from the stable onboard oscillator, guaranteeing a deterministic real-time clock. The hardware FSM controller ensures state transitions are glitch-free and independent of software overhead. Furthermore, a hardware implementation can be seamlessly integrated into a larger SoC (System-on-Chip) framework for multi-actuator smart home management. This project demonstrates how FPGA logic can address a real-world automation problem with precision and reliability that software alone cannot match.

Objectives

Primary Objective

To design and implement a hardware-based automated pet feeding controller using a Moore Finite State Machine (FSM) architecture on an FPGA, capable of real-time timekeeping, schedule-driven feeding automation, and manual override.

Specific Objectives

1. Develop a clock divider to derive a precise 1 Hz tick from the 100 MHz FPGA system clock.
2. Implement a real-time digital clock (HH:MM:SS) using cascaded register counters.
3. Design a schedule register bank to store enable/disable states for four distinct daily feeding times.
4. Implement a Moore FSM Control Unit to govern IDLE and FEEDING operational modes.
5. Build an ALU datapath for clock increments, time comparisons, and feeding countdown.
6. Drive a 7-segment display for real-time clock display and a FEED indicator.
7. Include a manual push-button override (btnC) for immediate feeding activation.

8. Verify system behavior through functional simulation using Vivado testbenches.

System Architecture

The proposed system is composed of four major hardware subsystems: the Clock Divider Module, the Memory Unit (register bank), the Arithmetic Logic Unit (ALU) Datapath, and the Control Unit (Moore FSM). Together, these components form a complete pipeline from system clock input to physical actuator output.

The Clock Divider generates a stable 1 Hz enable pulse from the 100 MHz FPGA oscillator by counting to 99,999,999 and toggling an enable flag. This pulse drives the timekeeping registers in the Memory Unit. The ALU continuously performs increment and comparison operations, feeding comparison results as flags to the Control Unit. The FSM uses these flags along with switch and button inputs to transition between IDLE and FEEDING states, producing output control signals for the motor driver, status LED, and 7-segment display decoder.

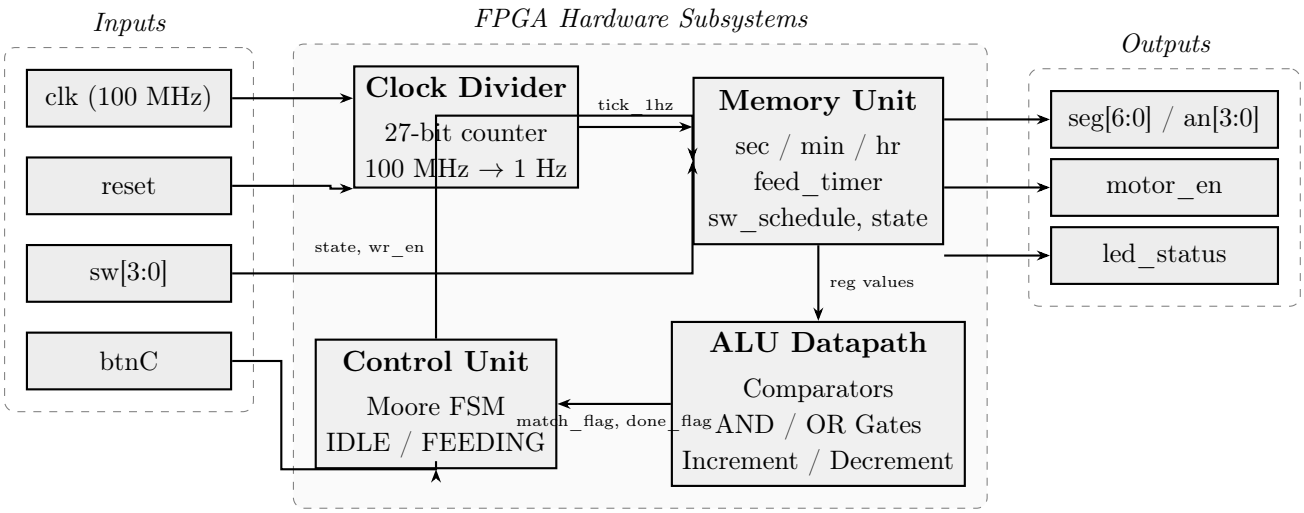

Figure 1: System Architecture Block Diagram of the Automated Pet Feeding Scheduler

Control Unit

The Control Unit is the central orchestrator of the system, implemented as a synchronous Moore FSM. In a Moore machine, all outputs are determined solely by the current state, ensuring glitch-free, stable output signals. It coordinates when the timekeeping registers increment, when the feeding motor activates, when the countdown timer runs, and which mode the 7-segment display decoder operates in. The FSM has exactly two states:

- **State 0 – IDLE:** The system continuously monitors the real-time clock against the four enabled target times. It increments the clock each second and drives the display in time-display mode. The motor output signal is held LOW.
- **State 1 – FEEDING:** Activated when a time-match is detected or the manual button is pressed. The motor output signal is asserted HIGH, the feeding countdown timer is loaded and decremented, and the display switches to FEED mode. The FSM automatically transitions back to IDLE when the countdown reaches zero.

Transition conditions:

- IDLE → FEEDING: Triggered when (`match_flag == 1`) OR (`btnC == 1`).
- FEEDING → IDLE: Triggered when (`feed_timer == 5'b00000`), i.e., countdown reaches zero.

The FSM is implemented with a 1-bit state register (`reg state`), updated on every positive clock edge. Output signals (`motor_en`, `display_mode`, `led_status`) are purely combinational functions of the current state, consistent with Moore machine behavior.

Memory Unit

The Memory Unit stores all persistent state information required for system operation. Unlike RAM-based storage, all registers are implemented as synthesized flip-flop arrays in Verilog using the `reg` keyword. The following table describes every register, its data type, bit width, and purpose:

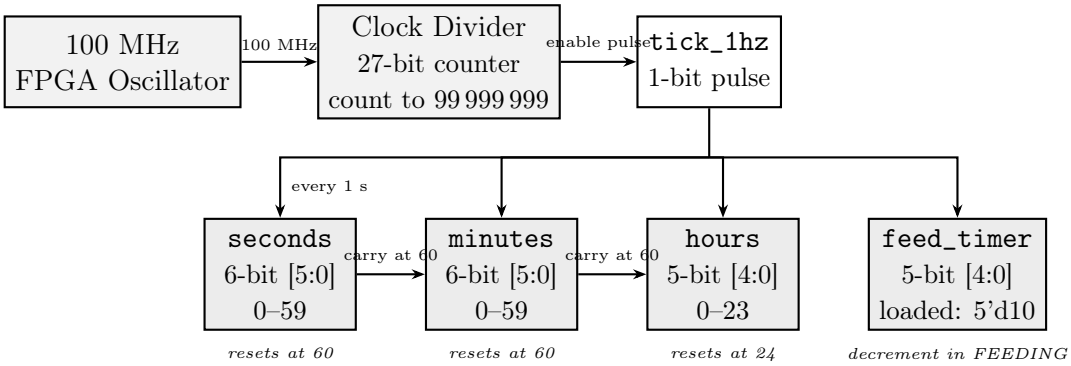

Figure 2: Clock Divider Chain and Timekeeping Register Cascade (Memory Unit)

Total register storage: $6 + 6 + 5 + 4 + 5 + 1 + 27 + 1 = 55$ flip-flops, plus the combinational wires for match flags and output signals. All registers are reset synchronously to their default values on the active-high reset signal.

Arithmetic Logic Unit (ALU)

The ALU Datapath performs all arithmetic and logical operations required by the scheduler. It operates continuously as combinational logic, producing result flags on every clock cycle that feed directly into the Control Unit FSM. The ALU performs the following operations:

Arithmetic Operations:

- **Clock Increment (Addition):** `seconds = seconds + 6'd1`. Performed every `tick_1hz` pulse. If `seconds` reaches 60, it resets to 0 and propagates a carry: `minutes = minutes + 6'd1`. Similarly, if `minutes` reaches 60, it resets and `hours = hours + 5'd1`. If `hours` reaches 24, it resets to 0. These cascaded additions form the complete timekeeping chain.
- **Countdown Decrement (Subtraction):** `feed_timer = feed_timer − 5'd1`. Executed every `tick_1hz` pulse while in FEEDING state. The 5-bit register supports values 0–31; it is loaded with `5'd10` on FEEDING entry and decremented until it reaches `5'b00000`, which triggers the IDLE transition.

Logical Operations:

Table 1: Memory Unit Register Description

Register Name	Data Type	Bit Width	Purpose
`seconds`	`reg`	6-bit [5:0]	Current seconds (0–59). Rolls over at 60, triggers minutes increment.
`minutes`	`reg`	6-bit [5:0]	Current minutes (0–59). Rolls over at 60, triggers hours increment.
`hours`	`reg`	5-bit [4:0]	Current hours (0–23). Rolls over at 24, resets to 0 (midnight wrap).
`sw_schedule[3:0]`	`reg`	4-bit [3:0]	Each bit enables one feeding slot: bit 0 = 08:00, bit 1 = 12:00, bit 2 = 17:00, bit 3 = 21:00. Set by FPGA slide switches.
`feed_timer`	`reg`	5-bit [4:0]	Countdown register loaded with `5'd10` (10 seconds) on FEEDING entry. Decremented each second. Feeding stops when it reaches 0.
`state`	`reg`	1-bit	FSM state register. 0 = IDLE, 1 = FEEDING.
`clk_div_counter`	`reg`	27-bit [26:0]	Free-running counter clocked at 100 MHz. Generates a 1 Hz enable tick when count reaches 99,999,999.
`tick_1hz`	`reg`	1-bit	One-clock-cycle wide pulse asserted every second. Drives all timekeeping register updates.

- **Equality Check (Time Match):** Four parallel 11-bit comparators check whether `{hours, minutes}` equals each target time: `11'b00010_000000` (08:00), `11'b01100_000000` (12:00), `11'b10001_000000` (17:00), and `11'b10101_000000` (21:00). Each comparator produces a 1-bit match result.
- **Bitwise AND (Schedule Gate):** Each match result is ANDed with its corresponding switch bit: `slot0_trigger = match_0800 & sw_schedule[0]`. This gates the time match through the user-configured schedule enable register, ensuring disabled slots never trigger feeding.
- **Logical OR (Trigger Aggregation):** The four gated slot triggers are ORed together with the manual button input to produce the final `match_flag`: `match_flag = slot0_trigger | slot1_trigger | slot2_trigger | slot3_trigger | btnC`. This single 1-bit flag drives the FSM state transition.
- **Equality Check (Countdown Zero):** A comparator checks `feed_timer == 5'b00000` to produce the `done_flag` that signals the FSM to exit FEEDING state.

All ALU operations on time registers use unsigned arithmetic. The 6-bit `seconds` and

`minutes` registers have sufficient range (0–63) to hold values up to 59 without overflow. The 5-bit `hours` register holds values 0–23 safely within its 0–31 unsigned range.

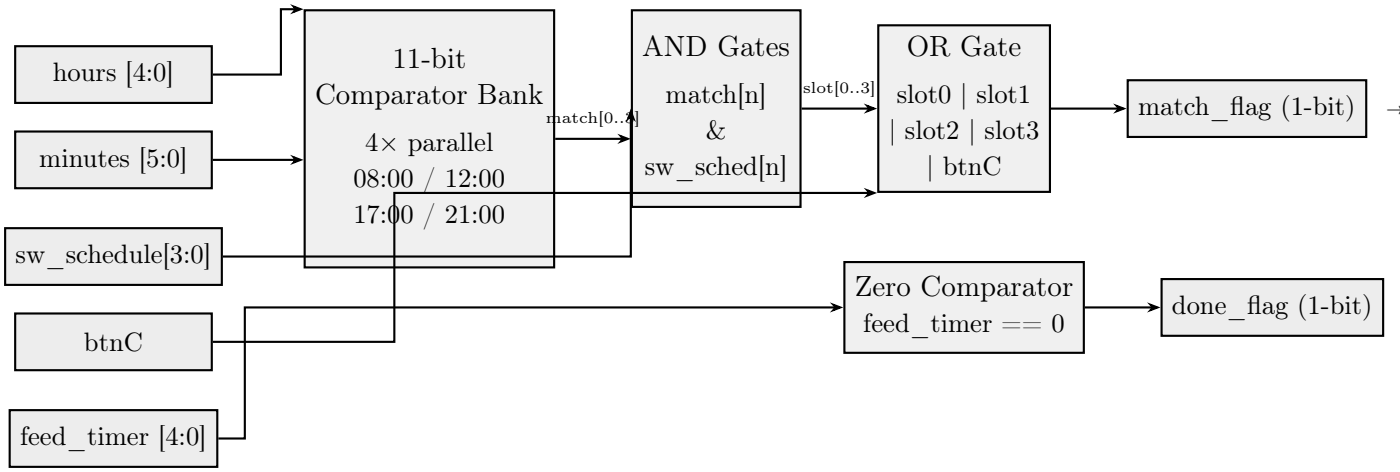

Figure 3: ALU Datapath Logic Diagram – Combinational Signal Flow

Finite State Machine Design

FSM States

The Control Unit Moore FSM consists of two well-defined operational states, each with distinct responsibilities and output assignments:

- **IDLE (State = `1'b0`):** The default quiescent state. The system increments the real-time clock once per second, continuously monitors all time-match comparators, and drives the 7-segment display in HH:MM mode. All actuator outputs remain deasserted. The system remains in IDLE until a `match_flag` is received.
- **FEEDING (State = `1'b1`):** The active feeding state. The motor enable signal is asserted, the feeding countdown timer counts down from 10, and the 7-segment display switches to FEED mode with the status LED illuminated. The system remains in FEEDING until the countdown timer reaches zero, after which it unconditionally transitions back to IDLE.

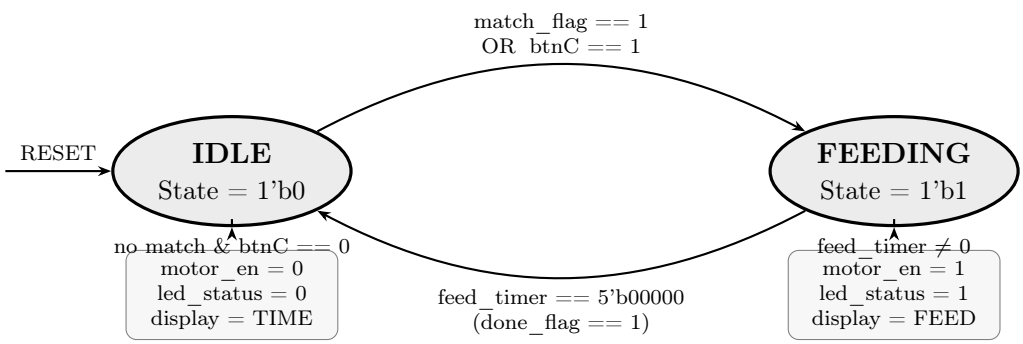

Figure 4: Moore FSM State Diagram with Output Annotations – Control Unit

State Transition Logic

Transitions are governed by two input flags derived from the ALU datapath and external input signals:

- **IDLE → FEEDING:** Transition occurs on the positive edge of `clk` when `match_flag == 1`. This flag is asserted when any enabled time slot matches the current HH:MM or when the manual override button `btnC` is pressed. The manual button acts as an immediate interrupt, bypassing all schedule checks.
- **FEEDING → IDLE:** Transition occurs when `done_flag == 1`, i.e., when `feed_timer` decrements to zero. This ensures the feeding duration is precisely 10 seconds regardless of how the FEEDING state was triggered. No external input can extend or abort the feeding cycle once initiated.

Because this is a Moore FSM, all outputs (`motor_en`, `display_mode`, `led_status`) change only on state transitions, guaranteeing stable, glitch-free signals to connected peripherals.

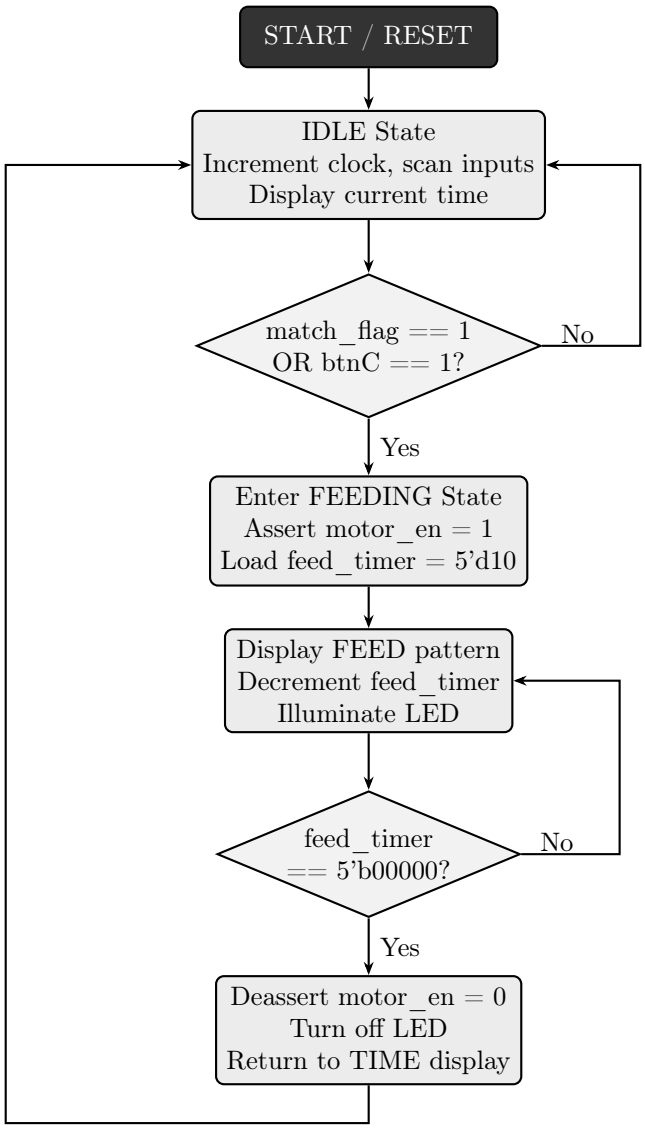

Figure 5: FSM Operation Flowchart – Full System Control Flow

Input and Output Signals

The system interfaces with the FPGA board through the following input and output signals:

Table 2: Input and Output Signal Description

Signal Name	Direction	Width	Description
`clk`	Input	1-bit	100 MHz onboard FPGA system clock. Source of all timing.
`reset`	Input	1-bit	Active-high synchronous reset. Clears FSM, all registers, and counters.
`sw[3:0]`	Input	4-bit	Slide switches. Each bit enables one feeding schedule slot (SW0=08:00, SW1=12:00, SW2=17:00, SW3=21:00).
`btnC`	Input	1-bit	Center push-button. Manual override for immediate feeding activation.
`seg[6:0]`	Output	7-bit	Seven-segment display cathode signals. Encodes current time digits or FEED pattern.
`an[3:0]`	Output	4-bit	Anode select signals for 4-digit multiplexed 7-segment display.
`motor_en`	Output	1-bit	Motor enable signal. HIGH during FEEDING state; drives feeder actuator.
`led_status`	Output	1-bit	Status LED. OFF during IDLE (monitoring); ON during FEEDING.

Reset and Constraint Mechanisms

The system employs a synchronous, active-high global reset signal to establish safe, deterministic initialization. On assertion of reset:

- All time registers (`seconds`, `minutes`, `hours`) are cleared to `6'b000000`, `6'b000000`, and `5'b00000` respectively, initializing the clock to 00:00:00.
- The FSM state register is cleared to `1'b0` (IDLE).
- The feeding countdown register `feed_timer` is cleared to `5'b00000`.
- The clock divider counter `clk_div_counter` is cleared to `27'b0`, restarting the 1 Hz generation chain.
- All output signals (`motor_en`, `led_status`, `display_mode`) are deasserted to safe inactive levels.

Overflow protection is inherent in the design: the `seconds` and `minutes` registers are gated with explicit rollover conditions (`== 6'd60`) rather than relying on natural binary

overflow, preventing any timing errors. The `hours` register uses a 24-hour modulo condition (`== 5'd24`). The `feed_timer` countdown is bounded within its 5-bit range by stopping decrement when it reaches zero, preventing underflow.

Hardware Implementation

The design was developed using Register-Transfer Level (RTL) design methodology in Verilog HDL within the Xilinx Vivado Design Suite. The target hardware is a Basys 3 FPGA development board featuring the Artix-7 FPGA (XC7A35T). The implementation flow encompasses RTL design, functional simulation, synthesis, and bitstream generation for download to the physical FPGA board.

The top-level module instantiates four submodules: the Clock Divider, the Memory/Register Unit, the ALU Datapath, and the FSM Control Unit. A separate 7-segment display controller module handles multiplexed display driving, cycling through the four display digits at approximately 1 kHz to exploit persistence of vision. A custom segment decoder translates BCD digit values (0–9) to 7-segment cathode patterns, and a dedicated FEED pattern (hardcoded as `7'b0111000` for 'F', `7'b0110000` for 'E', etc.) is output when in FEEDING state.

The XDC constraint file maps all I/O ports to the physical pin locations on the Basys 3 board: the 100 MHz system clock to W5, reset to U18 (BTNC on some variants), slide switches to V17–V10, and the 7-segment display anodes and cathodes to their respective Basys 3 pin assignments.

Testing and Verification

The system was verified using automated Verilog testbenches (`tb_pet_feeder.v`) developed within Vivado's built-in simulator. To accelerate simulation without modifying RTL source code, the testbench overrides the clock divider threshold parameter to generate a 1 Hz tick every 10 ns (simulation time) rather than waiting for real 100 MHz cycles.

Test cases included:

1. **Normal Schedule Trigger:** Clock was fast-forwarded to 08:00:00 with SW[0] = 1. Verification confirmed `motor_en` asserted, `led_status` lit, display switched to FEED mode, and system returned to IDLE after 10 simulated seconds.
2. **Disabled Schedule:** Clock advanced to 12:00:00 with SW[1] = 0. Verified that no feeding trigger occurred, confirming the AND gate schedule-gating logic.
3. **Manual Override:** `btnC` was asserted at an arbitrary time. Verified immediate IDLE → FEEDING transition regardless of current time or switch states.
4. **Countdown Accuracy:** Verified `feed_timer` decremented from `5'd10` to `5'd0` in exactly 10 clock ticks (simulated seconds), with FEEDING → IDLE transition occurring precisely at zero.
5. **Rollover Verification:** Seconds incremented from 59 to 0 and propagated carry to minutes; minutes from 59 to 0 propagated to hours; hours from 23 to 0 correctly rolled over to midnight.
6. **Reset Behavior:** Reset asserted mid-FEEDING; all registers confirmed cleared to initial values and FSM confirmed in IDLE state on next clock cycle.

Limitations and Future Work

While the current implementation achieves all core design objectives, several limitations exist:

- The four feeding schedule times are hardcoded at 08:00, 12:00, 17:00, and 21:00. Reprogramming requires an RTL modification and re-synthesis. A future version could allow runtime-configurable schedule times via a UART interface or onboard button-based input.
- The feeding duration is fixed at 10 seconds. Parameterizing this value and allowing switch-selectable durations would add flexibility.
- The current design lacks persistent time storage: a power cycle resets the clock to 00:00:00. Integration of an external RTC (Real-Time Clock) module such as DS1307 over I2C would provide battery-backed time retention.
- The 7-segment display is limited to a 4-digit multiplexed interface; a future iteration could incorporate an LCD or OLED module for richer status information.
- Portion control (dispensing a specific food weight) is not implemented; motor drive duration is used as a proxy. A load-cell-based feedback loop could be added for accurate portioning.

Conclusion

The FPGA-based Automated Pet Feeding Scheduler successfully demonstrates a complete hardware digital system design, integrating a clock divider, a real-time register-based timekeeping unit, a schedule-gated ALU comparator datapath, and a Moore FSM Control Unit. The decoupling of the memory unit, the ALU, and the FSM into distinct, well-defined hardware modules confirms highly deterministic behavior that reliably outperforms equivalent software executing on a sequential microcontroller. The project validates the power of FPGAs in real-time embedded automation: hardware-level timing, glitch-free state transitions, and configurable I/O make the Automated Pet Feeding Scheduler a robust, extensible platform for smart home applications.

References

[1] M. M. Mano and M. D. Ciletti, *Digital Design*, 6th ed. Pearson, 2017.

[2] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed. McGraw-Hill, 2013.

[3] J. F. Wakerly, *Digital Design: Principles and Practices*, 4th ed. Pearson, 2005.

[4] Xilinx Inc., *Vivado Design Suite User Guide: Getting Started (UG910)*, 2019.

[5] Digilent Inc., *Basys 3 FPGA Board Reference Manual*, 2021.